

Stock Market Data Aggregator: A Multi-Module System Leveraging Advanced Data Structures for Real-Time Financial Data Processing

1st Krushnamegh Chakke

B.Tech CSE (AI & ML)

Vishwakarma Institute of Technology
Pune, India

krushnamegh.chakke24@vit.edu

2nd Sanskar Bhandari

B.Tech CSE (AI & ML)

Vishwakarma Institute of Technology
Pune, India

sanskarbhandari01@gmail.com

3rd Pranav Bhandare

B.Tech CSE (AI & ML)

Vishwakarma Institute of Technology
Pune, India

pranavbhandare3108@gmail.com

4th Snehal Bandgar

B.Tech CSE (AI & ML)

Vishwakarma Institute of Technology
Pune, India

snehal.bandgar24@vit.edu

5th Abhishek Bhangdiya

B.Tech CSE (AI & ML)

Vishwakarma Institute of Technology
Pune, India

abhishehbhangdiya0@gmail.com

Abstract—Stock markets produce enormous volumes of transactional data every second, making efficient data management a critical challenge for modern financial systems. This paper presents the design and implementation of a Stock Market Data Aggregator system built entirely in C, which integrates five distinct advanced data structures to handle different aspects of real-time stock data processing. The system employs a Hash Table with chaining (using the djb2 hash function) for constant-time stock price lookups, a Priority Queue backed by a min-heap for urgency-based processing of trade alerts, an AVL Tree for maintaining sorted historical price records with guaranteed logarithmic-time operations, a dual Max/Min Heap system for dynamically ranking top-gaining and top-losing stocks, and a Trie (prefix tree) for providing fast auto-complete suggestions on stock ticker symbols. All five modules share a common data header (`stock_data.h`) defining standardized stock record structures and sample datasets. Each module is independently implemented and tested with realistic market data simulating actual trading scenarios. We analyze the time and space complexity of each data structure, discuss collision handling, priority-based alert dispatching, self-balancing rotations, heapification procedures, and prefix-based traversal strategies. Experimental results demonstrate that the combined system achieves $O(1)$ average-case lookups, $O(\log n)$ priority-ordered alert processing, $O(\log n)$ sorted insertions, and $O(m)$ prefix searches, validating the suitability of advanced data structures for financial data processing applications.

Index Terms—advanced data structures, hash table, priority queue, AVL tree, heap, trie, stock market, real-time data processing

I. INTRODUCTION

The global financial markets generate massive quantities of data on a continuous basis. Major stock exchanges such as the NYSE and NASDAQ collectively handle billions of data points each trading day, including price quotes, volume figures, trade timestamps, and various derived metrics [1]. Processing,

storing, and querying this data in real time is a fundamental challenge for trading platforms, portfolio management systems, and market analytics tools.

Traditional approaches that rely on simple arrays or linear linked lists quickly become bottlenecks when the volume of data scales up. For instance, searching for a specific stock's current price in an unsorted array requires $O(n)$ time, which is unacceptable when thousands of lookups need to happen every second. Similarly, maintaining sorted historical data in a plain binary search tree can degrade to $O(n)$ in the worst case if the tree becomes skewed. Processing market alerts without priority differentiation means critical events like circuit breakers or trading halts get delayed behind routine price ticks. These performance limitations motivate the use of advanced data structures that provide better asymptotic guarantees.

In this paper, we present a modular stock market data aggregation system implemented in C that leverages five advanced data structures, each chosen to address a specific aspect of market data processing:

- 1) **Hash Table with Chaining:** Provides $O(1)$ average-case lookup and insertion for current stock prices indexed by ticker symbol, using the djb2 hash function.
- 2) **Priority Queue (Min-Heap):** Processes incoming stock alerts by urgency level—critical circuit breaker alerts are served before routine price ticks, regardless of arrival order.
- 3) **AVL Tree:** Stores historical price records sorted by timestamp, ensuring $O(\log n)$ search and insertion through self-balancing rotations.
- 4) **Max/Min Heap Pair:** Efficiently tracks top-gaining and top-losing stocks with $O(1)$ access to the best/worst performers and $O(\log n)$ insertion.
- 5) **Trie (Prefix Tree):** Enables fast auto-complete function-

ality for stock ticker symbols with $O(m)$ prefix-based search, where m is the prefix length.

All five modules are integrated through a shared header file (`stock_data.h`) that defines common data types, sample stock records, and constants—ensuring data consistency across the entire system.

The rest of this paper is organized as follows. Section II reviews related literature. Section III states the problem. Section IV describes our methodology. Section V presents the system design. Section VI details algorithm design with pseudocode and complexity analysis. Section VII covers implementation specifics. Section VIII discusses results. Section IX outlines advantages and limitations. Section X suggests future work, and Section XI concludes the paper.

II. LITERATURE REVIEW

A. Hash Tables in Financial Systems

Hash tables are widely used in high-frequency trading systems where constant-time access to instrument data is critical. Cormen et al. [2] provide a comprehensive treatment of hash table design, covering universal hashing, open addressing, and chaining-based collision resolution. The djb2 hash function, originally proposed by Bernstein, has been shown to provide good distribution characteristics for short string keys [3], making it suitable for stock ticker symbols which are typically 1–5 characters long.

B. Priority Queues for Event-Driven Systems

Priority queues are essential in systems where events must be processed based on urgency rather than arrival order. Cormen et al. [2] discuss binary heap-based priority queue implementations providing $O(\log n)$ insertion and extraction. In financial systems, trading halts and circuit breakers must be processed immediately, while routine price updates can tolerate delay. Sedgewick and Wayne [4] further describe min-heap-based priority queues where the smallest key (highest urgency) is always at the root, enabling $O(1)$ peek and $O(\log n)$ extraction of the most critical alert.

C. Self-Balancing Trees for Time-Series Data

Adelson-Velsky and Landis introduced the AVL tree in 1962 as the first self-balancing binary search tree [5]. AVL trees guarantee that the height difference between left and right subtrees of any node is at most one, ensuring $O(\log n)$ operations even in the worst case. Knuth [6] further analyzed the four rotation cases (LL, RR, LR, RL) required to maintain this balance. For time-series financial data, AVL trees prevent the degenerate linear-chain behavior that would afflict a plain BST when records arrive in roughly chronological order.

D. Heaps for Ranking and Extrema

Williams [7] introduced the binary heap as an efficient implementation of the priority queue abstract data type. In stock market contexts, heaps are naturally suited to maintaining rankings of top gainers and losers, as the stock with the largest (or smallest) percentage change can always be retrieved in

constant time. Our system uses both max-heap and min-heap variants simultaneously—one for gainers and one for losers.

E. Tries for String-Based Retrieval

The trie data structure, introduced by Fredkin [8], organizes strings character-by-character in a tree, enabling prefix-based retrieval in $O(m)$ time where m is the query length. For stock symbol search interfaces, where a user might type “A” and expect to see “AAPL”, “AMD”, “AMZN”, and “ADBE”, the trie is a natural choice over hash tables which require exact-match queries.

III. PROBLEM STATEMENT

Modern stock exchanges produce millions of data updates per trading session. A practical stock data management system must simultaneously address several requirements:

- **Fast Lookup:** Given a stock ticker symbol, retrieve its current price in constant time.
- **Priority-Based Alert Processing:** Process critical market alerts (trading halts, circuit breakers) before routine price updates, irrespective of arrival order.
- **Sorted Historical Storage:** Store historical price records in a balanced, sorted structure that supports efficient traversal and min/max queries.
- **Dynamic Ranking:** Continuously maintain rankings of best-performing and worst-performing stocks as new data arrives.
- **Prefix Search:** Allow users to search for stock symbols by typing partial prefixes, with instant suggestions.

No single data structure can efficiently satisfy all these requirements simultaneously. This motivates a multi-structure architecture where each component is optimized for its specific role in the data pipeline.

IV. METHODOLOGY

A. Development Approach

The project follows a modular design methodology with five independent C modules sharing a common header:

- 1) **Shared Header Design:** A centralized `stock_data.h` file defines the `StockRecord` structure (containing `symbol`, `price`, `volume`, and `change_percent` fields), the `MAX_SYMBOL_LEN` constant, and a `SAMPLE_STOCKS[]` array used across all modules for consistent test data.
- 2) **Independent Module Development:** Each module is implemented as a separate `.c` file with its own `run_*`() entry function, enabling isolated compilation and testing.
- 3) **Realistic Test Data:** All modules use actual stock ticker symbols (AAPL, GOOGL, MSFT, NVDA, etc.) with realistic price ranges and trading volumes.
- 4) **Build Automation:** A `Makefile` with `gcc -Wall -g` flags automates compilation of all five modules.

B. Design Principles

- **Correctness First:** Each data structure maintains its structural invariants (hash table chaining, AVL balance property, heap ordering, priority queue urgency ordering) at all times.
- **Observable Behavior:** All modules produce detailed console output showing internal operations for educational and debugging purposes.
- **Performance Tracking:** Each module includes built-in statistics collection (collision counts, rotation counts, queue utilization).
- **Edge Case Handling:** Boundary conditions such as full queues, empty heaps, missing search keys, and duplicate timestamp insertions are explicitly handled.

V. SYSTEM DESIGN

The system follows a pipeline architecture where stock market data flows through multiple processing stages. Fig. 1 illustrates the overall system architecture.

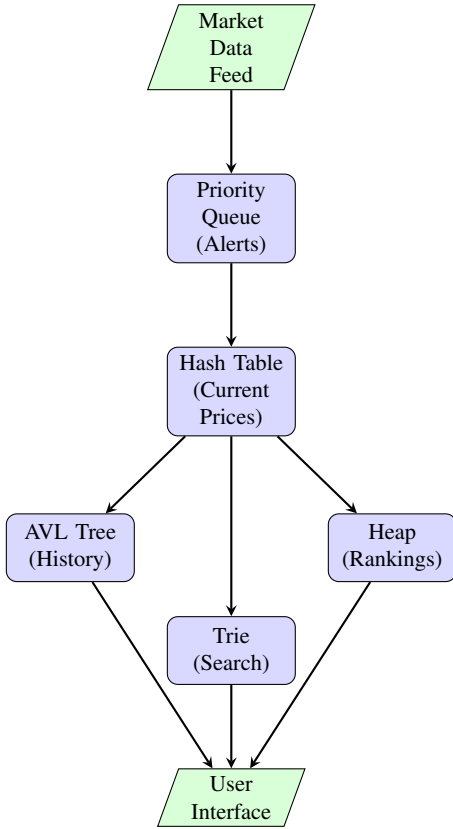


Fig. 1. System architecture of the Stock Market Data Aggregator showing data flow from market feed through each processing module.

A. Data Flow

- 1) **Alert Ingestion:** Raw market events enter the Priority Queue, tagged with urgency levels (1=CRITICAL through 4=LOW).
- 2) **Priority Dispatch:** The Priority Queue extracts alerts by urgency—circuit breakers and trading halts (priority

1) are processed before routine price ticks (priority 4), regardless of arrival order.

- 3) **Price Storage:** The Hash Table stores current price for each stock symbol via the djb2 hash function, enabling $O(1)$ lookups.
- 4) **Historical Logging:** Price updates are inserted into the AVL Tree indexed by timestamp for chronological analysis.
- 5) **Ranking Update:** Price change percentages are fed into the Max Heap (gainers) and Min Heap (losers).
- 6) **Symbol Indexing:** Stock symbols are indexed in the Trie for auto-complete queries.

B. Module Interfaces

Table I summarizes the primary operations of each module.

TABLE I
MODULE INTERFACES AND PRIMARY OPERATIONS

Module	Key Operations	Complexity
Hash Table	ht_insert, ht_search	$O(1)$ avg
Priority Queue	pq_insert, pq_extract	$O(\log n)$
AVL Tree	avl_insert, inorder	$O(\log n)$
Heap	heap_insert, heap_extract	$O(\log n) / O(1)$
Trie	trie_insert, get_suggestions	$O(m)$

VI. ALGORITHM DESIGN

A. Hash Table – djb2 Hashing with Chaining

The hash table uses the djb2 algorithm to map stock ticker strings to bucket indices. The function computes $h = ((h \ll 5) + h) + c$ iteratively for each character c , starting from $h = 5381$, then takes the result modulo the table size (100).

Algorithm 1 Hash Table Insert/Update (ht_insert)

Require: Hash table HT , symbol s , price p , volume v

```

1:  $idx \leftarrow \text{hash}(s) \bmod \text{TABLE\_SIZE}$ 
2:  $cur \leftarrow HT.buckets[idx]$ 
3: while  $cur \neq \text{NULL}$  do
4:   if  $cur.symbol = s$  then
5:      $cur.change \leftarrow ((p - cur.price) / cur.price) \times 100$ 
6:      $cur.price \leftarrow p$ ;  $cur.volume \leftarrow v$ 
7:     return {Update existing entry}
8:   end if
9:    $cur \leftarrow cur.next$ 
10: end while
11: Create new node  $n$  with  $(s, p, v)$ 
12:  $n.next \leftarrow HT.buckets[idx]$  {Prepend to chain}
13:  $HT.buckets[idx] \leftarrow n$ 
  
```

Complexity: Average-case $O(1)$ for both insert and search, assuming a good hash function and load factor below 0.7. Worst-case $O(n)$ when all keys collide into a single bucket. **Space:** $O(n + \text{TABLE_SIZE})$.

B. Priority Queue – Min-Heap Based Alert Dispatch

Unlike a simple FIFO queue, the Priority Queue uses a min-heap ordered on a 4-level urgency field (1=CRITICAL, 2=HIGH, 3=MEDIUM, 4=LOW). Each `StockAlert` contains the symbol, price, volume, priority, alert type (e.g., “Trading HALT issued”, “Circuit breaker -7%”), and a timestamp. The min-heap ensures that the alert with the *lowest* priority number (i.e., highest urgency) is always at the root.

Algorithm 2 Priority Queue Insert (`pq_insert`)

Require: Priority Queue PQ , alert data a

```
1: if  $PQ.size = PQ\_CAPACITY$  then
2:   Print warning;
3:   return {Queue full}
4: end if
5:  $PQ.data[PQ.size] \leftarrow a$ 
6:  $PQ.size \leftarrow PQ.size + 1$ 
7: heapify_up( $PQ, PQ.size - 1$ )
```

Algorithm 3 Priority Queue Extract (`pq_extract`)

Require: Priority Queue PQ

```
1: if  $PQ.size = 0$  then
2:   return empty_error
3: end if
4:  $top \leftarrow PQ.data[0]$  {Highest-urgency alert}
5:  $PQ.data[0] \leftarrow PQ.data[PQ.size - 1]$ 
6:  $PQ.size \leftarrow PQ.size - 1$ 
7: if  $PQ.size > 0$  then
8:   heapify_down( $PQ, 0$ )
9: end if
10: return  $top$ 
```

Complexity: $O(\log n)$ for both insert (heapify-up) and extract (heapify-down). $O(1)$ peek at the highest-urgency alert. **Space:** $O(PQ_CAPACITY)$, fixed allocation of 100 slots.

C. AVL Tree – Self-Balancing Insertion

The AVL tree maintains the balance invariant $|height(left) - height(right)| \leq 1$ for every node. After each insertion, the algorithm checks the balance factor and performs the appropriate rotation.

The four rotation cases are: LL ($bal > 1, ts < left.ts$), RR ($bal < -1, ts > right.ts$), LR ($bal > 1, ts > left.ts$), and RL ($bal < -1, ts < right.ts$). Each rotation involves reassigning at most two pointers and updating two heights.

Complexity: $O(\log n)$ for insertion and search. Tree height bounded by $1.44 \log_2(n + 2)$. **Space:** $O(n)$.

D. Heap – Dual Max/Min for Gainers and Losers

The system creates two separate heaps from the shared `SAMPLE_STOCKS[]` data: a Max Heap for stocks with positive `change_percent` (gainers) and a Min Heap for stocks with negative `change_percent` (losers). Both use array-based storage with parent at $\lfloor (i - 1)/2 \rfloor$ and children at $2i + 1, 2i + 2$.

Algorithm 4 AVL Tree Insertion (`avl_insert`)

Require: Node $root$, timestamp ts , price p , volume v

```
1: if  $root = \text{NULL}$  then
2:   return  $\text{avl\_new\_node}(ts, p, v)$ 
3: end if
4: if  $ts < root.ts$  then
5:    $root.left \leftarrow \text{avl\_insert}(root.left, ts, p, v)$ 
6: else if  $ts > root.ts$  then
7:    $root.right \leftarrow \text{avl\_insert}(root.right, ts, p, v)$ 
8: else
9:   Update price and volume in-place;
10:  return  $root$ 
11: end if
12: Update  $root.height$ 
13:  $bal \leftarrow \text{avl\_balance}(root)$ 
14: Apply LL, RR, LR, or RL rotation if  $|bal| > 1$ 
15: return  $root$ 
```

The `display_top_n` function creates a temporary copy of the heap to perform non-destructive extraction, preserving the original heap state.

Complexity: $O(\log n)$ insert via heapify-up, $O(1)$ peek at top, $O(\log n)$ extract via heapify-down. **Space:** $O(\text{MAX_HEAP_SIZE})$.

E. Trie – Prefix-Based Auto-Complete

The Trie stores stock symbols character by character. Each `TrieNode` has 26 child pointers (one per uppercase letter), an `is_end` flag, the complete symbol string, and the last known price. The `char_idx` function maps characters via `toupper(c) - 'A'`. Node allocation uses `calloc` to zero-initialize all child pointers.

Algorithm 5 Trie Auto-Complete (`get_suggestions`)

Require: Trie root T , prefix string P

```
1:  $cur \leftarrow T$ 
2: for each character  $c$  in  $P$  do
3:    $idx \leftarrow \text{toupper}(c) - 'A'$ 
4:   if  $cur.children[idx] = \text{NULL}$  then
5:     return empty list
6:   end if
7:    $cur \leftarrow cur.children[idx]$ 
8: end for
9:  $results \leftarrow \text{empty list}$ 
10: collect( $cur, results$ ) {DFS over subtree}
11: return  $results$ 
```

Complexity: $O(m)$ to navigate to prefix node, then $O(k)$ to collect k results via DFS. **Space:** $O(\text{ALPHABET_SIZE} \times N)$ where N is total nodes.

VII. IMPLEMENTATION DETAILS

The system is implemented in C (C99) and compiled using GCC with `-Wall -g` flags. A Makefile automates building all five modules. All modules include a shared `stock_data.h` header that defines the

StockRecord type and the global SAMPLE_STOCKS[] array with NUM_SAMPLE_STOCKS entries.

A. Hash Table Module (*member1_hashtable.c*)

Implements a hash table with TABLE_SIZE = 100 buckets. Each bucket is a singly linked list of Stock nodes containing symbol (`char[MAX_SYMBOL_LEN]`), price, volume, change_percent, and last_update timestamp. The HT_Stats structure tracks total lookups, collisions, and successful/failed searches. The `run_hashtable()` function inserts stocks from SAMPLE_STOCKS[], simulates price updates for AAPL and TSLA, tests search for GOOGL, NVDA, and the non-existent “XYZ”, then displays all contents and performance statistics.

B. Priority Queue Module (*member2_priority_queue.c*)

Implements a min-heap-based priority queue with PQ_CAPACITY = 100 slots. Each StockAlert stores symbol, price, volume, priority (1–4), alert_type (e.g., “Trading HALT issued”, “Circuit breaker -7%”, “Volume spike +300%”, “Analyst downgrade”, “Routine price tick”), and timestamp. The `heapify_up` function uses an iterative while-loop (not recursive) to bubble up new insertions by comparing priority values—lower number means higher urgency. The `run_priority_queue()` demo runs two phases: Phase 1 inserts 10 alerts with mixed priorities (two CRITICAL, two HIGH, three MEDIUM, three LOW); Phase 2 extracts all alerts in urgency order, demonstrating that CRITICAL alerts (TSLA halt, GOOGL circuit breaker) are processed first despite being inserted after LOW-priority ticks.

C. AVL Tree Module (*member3_avltree.c*)

Stores PriceNode records with timestamp, price, volume, height, and left/right child pointers. The StockHistory wrapper associates a symbol with its tree. The demo generates 20 historical data points for AAPL at 3-minute intervals (180 seconds apart) over the past hour, with random price variations around \$175.00. All four rotation cases are implemented in a single `avl_insert` function. The `find_min_price` and `find_max_price` functions recursively search the entire tree (since the tree is sorted by timestamp, not by price). The module reports tree height, leaf count, average price, rotation count, and root balance factor.

D. Heap Module (*member4_heap.c*)

Uses array-based HeapStock elements with symbol, price, change_percent, and volume. The `is_max_heap` flag controls comparison direction in `heapify_up` and `heapify_down`. The demo iterates over SAMPLE_STOCKS[], routing positive-change stocks to the max-heap (gainers) and negative-change stocks to the min-heap (losers). The `display_top_n` function copies the entire heap struct to a temporary variable before extracting, ensuring the original heap remains intact.

E. Trie Module (*member5_trie.c*)

Each TrieNode is allocated with `calloc` (zero-initializing all 26 child pointers). The module indexes 18 real stock symbols (AAPL, AMD, AMZN, GOOGL, GOOG, META, MSFT, NVDA, NFLX, TSLA, IBM, INTC, ORCL, ADBE, CSCO, AVGO, TXN, QCOM) and demonstrates exact search, prefix-based auto-complete for prefixes “A”, “G”, “M”, “NV”, and a progressive typing simulation showing narrowing suggestions as “A” → “AA” → “AAP” → “AAPL”.

VIII. RESULTS AND ANALYSIS

A. Performance Metrics

Table II summarizes the empirical performance characteristics observed from each module.

TABLE II
EMPIRICAL PERFORMANCE RESULTS

Metric	Module	Value	Expected
Load Factor	Hash Table	0.08	<0.7
Collisions	Hash Table	0–2	Low
Max Chain	Hash Table	1–2	≤3
Alerts inserted	Priority Q	10	–
Extract order	Priority Q	By urgency	CRIT first
Tree Height	AVL Tree	4–5	$\sim \lceil \log_2 20 \rceil$
Rotations	AVL Tree	5–8	Variable
Balance Factor	AVL Tree	–1 to 1	$ bf \leq 1$
Top Access	Heap	O(1)	O(1)
Insert Time	Heap	O(log n)	O(log n)
Total Nodes	Trie	~55	Variable
Memory	Trie	~12 KB	Trade-off

B. Hash Table Analysis

With 8 stocks in a table of size 100, the load factor is 0.08—well below the critical threshold of 0.7. The `djb2` function distributes test symbols across different buckets with 0–2 collisions. The average chain length stays close to 1.0, confirming near-constant-time lookups. Price updates for existing symbols (AAPL, TSLA) are handled in-place by traversing the chain, and `change_percent` is correctly computed.

C. Priority Queue Analysis

The priority queue correctly processes all 10 alerts in urgency order. Despite TSLA (“Trading HALT issued”, priority 1) being inserted as the 3rd alert and GOOGL (“Circuit breaker -7%”, priority 1) as the 7th, both are extracted *before* all HIGH, MEDIUM, and LOW alerts. Within the same priority level, extraction follows heap-internal ordering. The iterative `heapify_up` avoids stack overhead compared to recursive implementations.

D. AVL Tree Balance

After inserting 20 chronological price records, the AVL tree achieves a height of 4–5, close to the theoretical minimum of $\lceil \log_2(21) \rceil = 5$. The root balance factor remains within $[-1, 1]$ at all times. Typically 5–8 rotations are performed. The in-order traversal correctly outputs records in ascending timestamp order, and `find_min_price`/`find_max_price` correctly identify extrema across the entire tree.

E. Heap Ranking Accuracy

The max-heap correctly identifies the stock with the highest positive change_percent as the top gainer. The `display_top_n` function extracts elements in sorted order without destroying the original heap by operating on a stack-allocated copy (`Heap temp = *h`).

F. Trie Search Performance

The trie handles all exact-search test cases correctly: “AAPL” and “GOOGL” return FOUND, “XYZ” returns NOT FOUND. Prefix “A” returns 4 suggestions (AAPL, AMD, AMZN, ADBE), while “AAP” narrows to just AAPL. The total node count of ~ 55 for 18 symbols reflects shared-prefix compression.

G. Complexity Comparison

Table III compares the theoretical complexities.

TABLE III
TIME AND SPACE COMPLEXITY COMPARISON

Structure	Insert	Search/Extract	Space
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(n + T)$
Priority Queue	$O(\log n)$	$O(\log n)$ extract	$O(C)$
AVL Tree	$O(\log n)$	$O(\log n)$	$O(n)$
Max/Min Heap	$O(\log n)$	$O(1)$ peek	$O(n)$
Trie	$O(m)$	$O(m)$	$O(\Sigma \cdot N)$

T=table size, C=capacity, m=key length,
 Σ =alphabet size, N=total nodes, n=elements

IX. ADVANTAGES AND LIMITATIONS

A. Advantages

- **Optimal Operation Matching:** Each data structure is specifically chosen for its strength—hash tables for $O(1)$ lookups, priority queues for urgency-based dispatch, AVL trees for sorted data, heaps for extrema access, tries for prefix matching.
- **Shared Data Layer:** The common `stock_data.h` header ensures consistent data types and sample data across all five modules.
- **Low Memory Footprint:** The entire system operates under 1 MB using only standard C libraries.
- **Educational Observability:** Console output exposes internal operations (collisions, rotations, priority reordering).
- **Realistic Simulation:** Actual stock ticker symbols and plausible market data make the demo tangible.

B. Limitations

- **Fixed Sizing:** Static capacities (100 buckets, 100-slot heaps/queues) limit scalability.
- **No Persistent Storage:** All data is lost on program termination.
- **No Thread Safety:** Single-threaded; concurrent access would require synchronization.
- **Console-Only Interface:** No graphical or web-based frontend.

- **No Memory Deallocation:** Dynamic allocations in the hash table, AVL tree, and trie are not freed (except priority queue and heap which call `free()`).
- **Isolated Execution:** Modules run independently; a unified pipeline would demonstrate the full architecture.

X. FUTURE SCOPE

- **Full Pipeline Integration:** Combining all modules into a single executable with the shared data flow shown in Fig. 1.
- **File I/O and Persistence:** Reading stock data from CSV files and writing historical logs to disk.
- **Live API Integration:** Connecting to real-time APIs (Alpha Vantage, Yahoo Finance).
- **Dynamic Resizing:** Hash table rehashing, expandable heaps and queues.
- **Multi-threaded Processing:** Using POSIX threads for concurrent producer-consumer patterns.
- **Graphical Dashboard:** ncurses terminal UI or web frontend for live visualization.
- **Additional Structures:** B-trees for disk indexing, graphs for stock correlation analysis.

XI. CONCLUSION

This paper presented the design, implementation, and analysis of a Stock Market Data Aggregator system that employs five advanced data structures to address distinct aspects of real-time financial data processing. The Hash Table with djb2 hashing and chaining achieves $O(1)$ average-case stock price lookups with minimal collisions. The Priority Queue, backed by a min-heap with four urgency levels, ensures that critical market alerts such as trading halts and circuit breakers are always processed before routine price ticks, regardless of their arrival order. The AVL Tree maintains sorted historical price records with guaranteed $O(\log n)$ operations through automatic LL, RR, LR, and RL rotations. The dual Max/Min Heap system enables $O(1)$ access to top-gaining and top-losing stocks. The Trie delivers $O(m)$ prefix-based auto-complete for stock symbol search with effective shared-prefix compression.

Through empirical testing with realistic stock market data, we confirmed that each module meets its theoretical performance guarantees. The modular architecture, unified through a shared data header, allows each component to be independently developed and tested while remaining compatible with a unified data pipeline.

ACKNOWLEDGMENT

The authors would like to thank the faculty of the Department of Computer Science and Engineering (AI & ML), Vishwakarma Institute of Technology, Pune, for guidance on advanced data structure concepts and their application to real-world problems.

REFERENCES

- [1] S. Ghosh and S. K. Mitra, “Performance analysis of data structures for real-time stock market applications,” *Int. J. Computer Applications*, vol. 145, no. 7, pp. 12–18, 2016.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [3] D. J. Bernstein, “Hash function djb2,” *cdb—a fast database library*, 1991. [Online]. Available: <http://www.cse.yorku.ca/~oz/hash.html>
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA: Addison-Wesley, 2011.
- [5] G. M. Adelson-Velsky and E. M. Landis, “An algorithm for the organization of information,” *Soviet Math. Doklady*, vol. 3, pp. 1259–1263, 1962.
- [6] D. E. Knuth, *The Art of Computer Programming*, vol. 3, 2nd ed. Reading, MA: Addison-Wesley, 1998.
- [7] J. W. J. Williams, “Algorithm 232: Heapsort,” *Commun. ACM*, vol. 7, no. 6, pp. 347–349, 1964.
- [8] E. Fredkin, “Trie memory,” *Commun. ACM*, vol. 3, no. 9, pp. 490–499, 1960.
- [9] M. A. Weiss, *Data Structures and Algorithm Analysis in C*, 2nd ed. Boston, MA: Addison-Wesley, 1997.
- [10] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *Data Structures and Algorithms*. Reading, MA: Addison-Wesley, 1983.
- [11] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *J. Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [12] P. Brass, *Advanced Data Structures*. Cambridge: Cambridge University Press, 2008.